

Relatório de 2º Trabalho de Sistemas Operativos

Trabalho Realizado por:

David Ramalho Nº 57823 Diogo Tavares Nº 58049

Francisco Rodrigues Nº 59119

Introdução

Este trabalho tem como objetivo aprofundar os conhecimentos sobre gestão de memória e escalonamento de processos em sistemas operativos. Ele é dividido em duas partes:

A 1ª Parte foca na implementação de um simulador de gestão de memória com paginação e memória virtual que pode utilizar duas estratégias de alocação diferente para paginação, LRU e FIFO ou seja Least Recently Used e First In First Out.

A 2ª parte do trabalho integra o simulador de escalonamento realizado no 1º trabalho com o gestor de memória desenvolvido na Parte 1 deste trabalho e adicionando novos estados.

Parte 1: Simulador Gestor de Memória

Implementa um simulador de gestão de memória com paginação e memória virtual, usando dois algoritmos de substituição (FIFO e LRU).

Tendo como especificações da Memória Principal um Tamanho de 21 kB e um Tamanho de cada página/frame: 3 kB (7 frames no total).

Estruturas de Dados Principais

- Frame (Memória Principal):
 - **owner_pid**: PID do processo dono.
 - **owner_page_num**: Número da página do processo.
 - **fifo_load_time**: Tempo de carregamento (FIFO).
 - **lru_last_access_time**: Último acesso (LRU).



- Process (Controle de Processos):
 - **pid**: Identificador do processo
 - **memory_size**: Tamanho total em bytes
 - **num_pages**: N° de páginas (calculado)
 - **has_terminated**: Flag de término (SIGSEGV)
 - **page_table**: Tabela de páginas

Funcionalidades Principais

- Inicialização:
 - Memória principal resetada
 - Processos criados com base no input
 - Tabelas de páginas alocadas dinamicamente
- Simulação de Acessos:

```
// O loop itera sobre o array de execução (um par de cada vez)
for (int i = 0; i < num_instructions / 2; i++)
{
    // Obter a instrução atual do array de execução selecionado
    int current_pid = current_exec_input[i * 2];
    int address = current_exec_input[i * 2 + 1];

    // Validação do PID para evitar crash se o input tiver um PID inválido
    if (current_pid <= 0 || current_pid > num_processes)
```

- Processa pares (PID, endereço) sequencialmente.
- Verifica validade do endereço (SIGSEGV se inválido).

Gerenciamento de Page Fault:

Page Hit: Atualiza o ultimo frame com LRU se necessário

Page Fault:

```
288 else
289 { // Page Fault
290     int target_frame_id = find_free_frame(main_memory);
291     if (target_frame_id == -1)
292     { // Precisa de substituição
293         if (algorithm == FIFO)
294         {
295             target_frame_id = find_victim_frame_fifo(main_memory);
296         }
297         else
298         { // LRU
299             target_frame_id = find_victim_frame_lru(main_memory);
300         }
301         // Desalocar a página antiga
302         int victim_pid = main_memory[target_frame_id].owner_pid;
303         int victim_page_num = main_memory[target_frame_id].owner_page_num;
304         processes[victim_pid - 1].page_table[victim_page_num].frame_id = -1;
305     }
```



- **Algoritmos de substituição:**

- FIFO: `find_victim_frame_fifo()` (frame mais antigo)
- LRU: `find_victim_frame_lru()` (frame menos recentemente usado)

Tratamento de SIGSEGV:

```
261 if (address >= proc->memory_size)
262 {
263     sigsegv_flags[current_pid - 1] = 1; // Marcar para imprimir SIGSEGV
264     proc->has_terminated = 1;
265
266     // Libertar todos os frames do processo terminado
267     for (int f = 0; f < NUM_FRAMES; f++)
268     {
269         if (main_memory[f].owner_pid == proc->pid)
270         {
271             main_memory[f].owner_pid = -1;
272         }
273     }
274 }
```

Termina o processo e os frames imediatamente.

Parte 2: Simulador de Escalonamento e Memória

Combina o simulador de memória (Parte 1) com o escalonador do Trabalho 1, adicionando novos estados e instruções.

1. Estruturas de Dados Utilizadas

- **Enumerações:**

- **State:** Representa os estados de um processo.
- **ErrorType:** Representa os tipos de erro que um processo pode sofrer.

- **Structs:**

- **PageTableEntry:** Contém o `frame_id` para mapear uma página a um frame de memória.
- **Frame:** Representa um frame na memória principal com informações do processo e página associada.
- **Process:** Estrutura principal dos processos com atributos como `instr`, `pc`, `estado`, `page_table`, entre outros.
- **InputData:** Estrutura que representa os dados de entrada (programas) e seu tamanho.



- **Variáveis globais:**

- **processos[]**: Vetor de ponteiros para processos ativos.
- **main_memory[]**: Vetor de Frames de memória principal.
- **id_proximo_processo**: O identificador do próximo processo.
- **total_processos_ativos**: O N° total de Processos Ativos.
- **current_input**: O N° do Input atual.

Parte 2: Simulador de escalonamento e memória

1. Introdução

A 2ª parte do trabalho consiste em integrar o simulador de escalonamento desenvolvido no primeiro trabalho com o simulador de memória virtual paginada da Parte 1, utilizando exclusivamente o algoritmo de substituição LRU (Least Recently Used). O sistema simula o funcionamento de um escalonador de CPU combinado com a gestão de memória paginada, lidando com acessos válidos e inválidos à memória, substituição de páginas, e diferentes tipos de instruções e erros (como SIGSEGV, SIGILL, SIGEOF).

2. Arquitetura geral do simulador

O simulador foi desenvolvido em C e estruturado de forma modular para separar as diferentes responsabilidades do sistema operativo. A lógica central reside num loop principal que simula o avanço do tempo em instantes. A cada instante, o simulador processa as transições de estado, executa o escalonamento e gere os acessos à memória.

A arquitetura assenta nos seguintes componentes:

- **Gestor de Processos**: Responsável pela criação, destruição e gestão do estado dos processos.
- **Escalonador**: Implementa o algoritmo Round Robin para decidir que processo utiliza a CPU.
- **Gestor de Memória**: Simula a memória principal paginada, tabelas de páginas e o algoritmo de substituição LRU.
- **Motor de Simulação**: O ciclo principal que orquestra todos os outros componentes, avançando o tempo e mantendo a coerência do sistema.

3. Estruturas de Dados Principais

A simulação é suportada por um conjunto de estruturas de dados robustas que representam os componentes fundamentais de um sistema operativo.



- **Processo:** A estrutura central, equivalente ao PCB, que armazena toda a informação de um processo:
 - id, pc (Program Counter), estado (NEW, READY, etc.).
 - **instr, tamanho_programa:** O código a ser executado.
 - **quantum_usado, tempo_no_estado, bloqueado_restante:** Campos para a gestão do escalonamento e temporização.
 - **memory_size, num_pages:** Informação sobre o espaço de endereçamento virtual.
 - **page_table:** Um ponteiro para um array de PageTableEntry, representando a tabela de páginas do processo.
 - **last_error:** Armazena o tipo de erro que levou ao término do processo (e.g., SIGSEGV).
- **Frame:** Representa um frame na memória principal física.
 - owner_pid, owner_page_num: Identificam que página de que processo ocupa o frame.
 - lru_last_access_time: Guarda o instante do último acesso, essencial para o algoritmo LRU.
- **PageTableEntry:** Representa uma entrada na tabela de páginas de um processo.
 - frame_id: Indica o frame físico onde a página está carregada. Um valor de -1 significa que a página não está na memória (resultando num Page Fault).
- **Filas (Queue):** O sistema utiliza filas FIFO (implementadas num módulo queue.c separado) para gerir os conjuntos de processos nos estados NEW, READY e BLOCKED.

4. Funcionalidades e Lógica de Simulação

4.1 Ciclo de Simulação (main loop)

O motor da simulação é o loop main, que a cada instante de tempo executa um ciclo de operações bem definido para garantir a precisão da temporização.

A lógica dentro do loop for (int time...) foi estruturada em fases para evitar transições de estado ambíguas e garantir que as consequências de uma ação ocorram no instante correto.

1. **Atualização de Estados:** No início de cada "tick", os processos nas filas de espera (NEW e BLOCKED) têm os seus contadores de tempo atualizados. Aqueles que terminaram a sua espera são movidos para a fila READY.

2. **Processamento do Processo Anterior:** O processo que estava em RUNNING na iteração anterior é avaliado. Se o seu quantum esgotou ou se uma instrução o marcou para mudar de estado (e.g., BLOCKED ou EXIT), ele é movido para a fila apropriada e a CPU é libertada.
3. **Escalaonamento (Dispatch):** Se a CPU está livre, o escalonador retira o primeiro processo da fila READY e move-o para o estado RUNNING.
4. **Execução da instrução (executar_instrucao):** O processo RUNNING executa a instrução apontada pelo seu pc.
5. **Impressão:** Após a execução, a função imprimir_estado é chamada para gerar a linha de output correspondente ao estado final do sistema para aquele instante de tempo, incluindo a lista de frames de cada processo.
6. **Limpeza:** Os processos que já cumpriram o seu tempo no estado EXIT (3 instantes) são permanentemente removidos, e a memória que ocupavam (frames e estruturas) é libertada.

4.2 Gestão de Memória

A interação com a memória é gerida por um conjunto de funções auxiliares:

- **tratar_acesso_memoria:** Centraliza a lógica de acesso, tratando SIGSEGV, Page Hits e a sinalização de Page Faults.
- **encontrar_frame_livre:** Procura sequencialmente um frame desocupado na memória.
- **encontrar_vitima_lru:** Implementa o algoritmo Least Recently Used, encontrando o frame que foi acedido há mais tempo para ser substituído. A regra de desempate é o menor ID de frame.
- **libertar_memoria_processo:** Percorre a memória principal e liberta todos os frames pertencentes a um processo que terminou.

5. Conclusão

O simulador desenvolvido cumpre os requisitos propostos, integrando com sucesso um escalonador de processos com um gestor de memória virtual. A implementação de uma máquina de estados com uma ordem de operações rigorosa foi crucial para garantir a precisão da temporização e a correção do output. O projeto serviu como uma excelente ferramenta prática para consolidar os conceitos teóricos de escalonamento, paginação e memória virtual estudados na unidade curricular.